

MEDIA-INFORMANT

Installation Instructions

Hosting the web application on a Synology NAS
with DSM Web Station and PHP 8.4

Document type	Operator installation guide	Platform	Synology DSM + Web Station
Runtime	PHP 8.4 CLI and PHP-FPM	Database	SQLite 3 via sqlite3 + pdo_sqlite
Scope	NAS-side hosting prerequisites	Date	20 September 2026

This guide records the Synology-specific work required to host Media-Informant. The application can be copied onto the NAS and served by Web Station, but it will not use SQLite correctly until PHP 8.4 is pointed at the real extension modules and both **sqlite3** and **pdo_sqlite** are enabled on the Web Station PHP profile. Those last two items were the difference between an installed app and a working app.

Why this document exists

DSM ships PHP 8.4 with a CLI `php.ini` whose `extension_dir` points at `/usr/local/lib/php84/modules`. That directory is empty. The actual `sqlite3.so` and `pdo_sqlite.so` files live inside the PHP 8.4 package under `/volume1/@appstore/PHP8.4/...` CLI and Web Station also use separate configuration. Fixing one does not fix the other.

1. Prerequisites

Complete these items in Package Center and Control Panel before touching PHP configuration.

Item	Notes
SSH	Enable in Control Panel → Terminal & SNMP. You will edit files as a user with <code>sudo</code> .
Web Station	Required. Serves the application and owns the PHP-FPM profiles.
PHP 8.4 package	Install from Package Center. Requires DSM 7.2 or later. Do not confuse this with a Docker PHP image.
HTTP server backend	Apache HTTP Server 2.4 or Nginx, as selected in Web Station for the virtual host that will serve Media-Informant.
Shared folder	A web-accessible folder (for example <code>web</code> or a dedicated site folder) where the application files will live.
http group on media shares	Read-only if the app will only catalog files. Read/Write if PHP Tools (rename and other write operations) will be used. See Section 6.

2. Create and assign a PHP 8.4 profile

Web Station keeps PHP settings in named profiles. Media-Informant should run under its own PHP 8.4 profile so later package updates or other sites do not silently change its extensions.

Step 2.1 — Create the profile

Open **Web Station** from the DSM Main Menu. In the left pane choose **Script Language Settings**. Create a new profile (or edit the PHP 8.4 profile created for this site). Set the script language to **PHP 8.4**. Give it a name you will recognize later, for example *Media-Informant PHP 8.4*.

Step 2.2 — Attach the profile to the site

In **Web Service Portal** (or the virtual host that points at the Media-Informant document root), set the PHP profile to the one created above. Save. Until this is done, the site may still be running an older PHP profile that does not have SQLite enabled.

3. Place the application files

Copy the Media-Informant application into the document root of that virtual host. Typical location:

```
/volume1/web/media-informant/
```

Exact folder names vary by share layout. What matters is that the virtual host document root and the application directory match, and that the HTTP user can read the PHP files and write the SQLite database path the app uses.

At this stage the app may load in a browser and still fail as soon as it opens a database. That is expected. The next two sections are the required Synology-specific fix.

4. Correct PHP 8.4 CLI extension_dir

The stock PHP 8.4 CLI configuration on DSM points `extension_dir` at a directory that does not contain the modules. Leave the original line in the file as a comment so the default value is not lost.

Step 4.1 — Confirm where the modules actually are

SSH into the NAS and run:

```
ls -ld /usr/local/lib/php84 /usr/local/lib/php84/modules
ls -l /volume1/@appstore/PHP8.4/usr/local/lib/php84/modules/sqlite3.so \
    /volume1/@appstore/PHP8.4/usr/local/lib/php84/modules/pdo_sqlite.so
php84 --ini
```

Expected result: `/usr/local/lib/php84/modules` is empty. The two `.so` files exist under `/volume1/@appstore/PHP8.4/usr/local/lib/php84/modules/`. Note the **Loaded Configuration File** printed by `php84 --ini` — that is the CLI file to edit. On this installation it is:

```
/usr/local/etc/php84/cli/php.ini
```

Step 4.2 — Edit php.ini

Open that file as root. Comment the existing `extension_dir` line with a leading semicolon and add a line that points at the `@appstore` modules directory. Quotation marks around the path are optional.

```
;extension_dir = /usr/local/lib/php84/modules
extension_dir = /volume1/@appstore/PHP8.4/usr/local/lib/php84/modules
```

In the same file, or in `/usr/local/etc/php84/cli/conf.d/extension.ini` if you prefer a drop-in, add:

```
extension = sqlite3.so
extension = pdo_sqlite.so
```

Do not reference libsqlite3.so in php.ini

PHP loads `sqlite3.so` and `pdo_sqlite.so`. Those modules already link against the system `libsqlite3.so`. The ini setting `sqlite3.extension_dir` is only for SQLite plugin modules loaded later via `SQLite3::loadExtension()`. It is not how you point PHP at `libsqlite3`.

Step 4.3 — Verify CLI

```
php84 -m | grep -i sqlite
```

Success is both names listed, with no “Unable to load dynamic library” warning:

```
pdo_sqlite
sqlite3
```

CLI PHP rereads the ini on every invocation. No service restart is required for command-line checks.

5. Enable sqlite3 in Web Station (required for the app)

This is the step that made the already-installed application work with SQLite. Web Station PHP-FPM does **not** use the CLI `php.ini` edited in Section 4. The site profile must have the modules enabled on its own Extensions tab.

Step 5.1 — Open the profile editor

In the DSM portal, open **Web Station**. In the left navigation choose **Script Language Settings**. Select the PHP 8.4 profile created for Media-Informant (the new profile from Section 2). Click **Edit**.

Step 5.2 — Enable the two modules

Open the **Extensions** tab. The tab lists PHP modules and an enable/disable state for each. Enable both of the following:

Module	Required state	Purpose
sqlite3	Enabled	Native SQLite3 API used by the application.
pdo_sqlite	Enabled	PDO driver. Enable even if the app currently uses the native API, so either access path works.

Save the profile. Saving the profile in the UI is what reloads PHP-FPM for that site. If the application was already deployed, this save is the change that allows it to open SQLite correctly.

If the command-line service tools are missing

DSM does not always put `synoservice` / `synoservicectl` on PATH. You do not need them if you Save the profile in Web Station. If you prefer the shell, try `sudo /usr/syno/bin/synopkg restart WebStation` or `sudo systemctl restart pkgctl-WebStation`. PHP-FPM workers appear as `pkg-WebStation-php84@<uuid>.service`.

6. Shared-folder rights for the http group

Web Station PHP-FPM runs as the system user `http`, which is a member of the `http` group. Media files typically live on other shares such as `/volume1/Video` and `/volume1/Music`, owned by a DSM user (often `guest`) with a Synology ACL. Unix mode `777` is not enough: the `+` on `ls -l` means the ACL can still deny `http`.

Also add those share paths to **open_basedir** on the PHP 8.4 profile **Settings** tab (not the **Core** tab): switch from **Default** to **Customized** and append `:/volume1/Video:/volume1/Music` to the existing list. That only allows PHP to *see* the paths. The ACL below is what allows or blocks writes.

PHP Tools planned?	http group right on Video / Music	Why
No — catalog and play only	Read only	The app can scan and stream files. Rename, move, and other Tools writes will fail with “write access is not permitted.”
Yes — use the PHP Tools subset (rename and similar)	Read/Write	Tools must write the parent folder of each media file. Assign R/W to the http group (System internal user / group list).

Step 6.1 — Set the ACL in DSM

Control Panel → Shared Folder → select **Video** (repeat for **Music**) → Edit → Permissions. Switch the list to **System internal user** so the `http` group appears. Assign **Read only** or **Read/Write** per the table above. Check **Apply to this folder, sub-folders and files**. Save.

Step 6.2 — Confirm the ACL

```
sudo synoacldtool -get /volume1/Video
```

When PHP Tools are enabled you want a line equivalent to:

```
group:http:allow:rwxdDaARwc--:fd-- (level:0)
```

The same right must exist on the parent directory of any file Tools will rename. Do not run the web app via `sudo` and do not `chown` the whole media library to `http` if Plex or other packages also use those shares.

7. Confirm the application

After the Extensions tab is saved, reload Media-Informant in the browser. The app should now create or open its SQLite database without a missing-driver or undefined-class error.

Optional PHP check from a page in the site document root:

```
<?php
echo 'sqlite3: ' . (extension_loaded('sqlite3') ? 'yes' : 'no') . PHP_EOL;
echo 'pdo_sqlite: ' . (extension_loaded('pdo_sqlite') ? 'yes' : 'no') . PHP_EOL;
print_r(PDO::getAvailableDrivers());
```

Run that file through the website, not through `php84` on SSH, if you want to test the same PHP-FPM profile the app uses. Remove the file when you are done.

8. What will overwrite this work

These edits are on package-owned files. They are the correct place to change, but they are not permanent across updates.

Change	Lost when
CLI <code>extension_dir</code> and <code>extension = lines</code>	PHP 8.4 package update or Repair in Package Center.
Web Station profile extension checkboxes	Usually kept, but re-check after a PHP 8.4 or Web Station update.
Files under <code>/volume1/@appstore/PHP8.4/</code>	Any reinstall or upgrade of the PHP 8.4 package. Do not patch those .so files; point at them.
System <code>libsqlite3.so</code> under <code>/lib</code> or <code>/usr/lib</code>	DSM update. Do not edit those libraries.
<code>http</code> group ACL on Video / Music	Survives package updates. Recheck if the shared folder is recreated or permissions are reset.

After a PHP 8.4 update, re-run Section 4.3 and re-open the Extensions tab in Section 5. If `php84 -m` no longer lists `sqlite3`, the ini was replaced and the commented original line plus the `@appstore` path must be restored.

9. Operator checklist

1	Web Station and PHP 8.4 packages installed.
2	Dedicated PHP 8.4 profile created under Script Language Settings.
3	Virtual host / portal assigned that PHP 8.4 profile.
4	Application files deployed to the document root.
5	CLI <code>php.ini</code> <code>extension_dir</code> commented and redirected to <code>/volume1/@appstore/PHP8.4/usr/local/lib/php84/modules</code> .
6	CLI <code>php.ini</code> (or <code>conf.d</code> drop-in) loads <code>sqlite3.so</code> and <code>pdo_sqlite.so</code> .
7	<code>php84 -m</code> lists both <code>sqlite3</code> and <code>pdo_sqlite</code> .
8	In Web Station → Script Language Settings → PHP 8.4 profile → Extensions, <code>sqlite3</code> and <code>pdo_sqlite</code> are enabled.
9	Profile saved so PHP-FPM reloads.
10	Browser test of Media-Informant opens or creates the SQLite database without a driver error.
11	<code>open_basedir</code> on the profile Settings tab includes <code>/volume1/Video</code> and <code>/volume1/Music</code> (and the site root).
12	<code>http</code> group has Read only on those shares if Tools will not be used, or Read/Write if PHP Tools will rename or write files.
13	<code>synoacldtool -get</code> on the share shows <code>group:http</code> with the intended rights, applied to children.

End of guide. SQLite requires Sections 4 and 5. Media-library access and PHP Tools require Section 6: Read only for catalog-only installs, Read/Write on the `http` group when Tools will rename files.